Transcrição

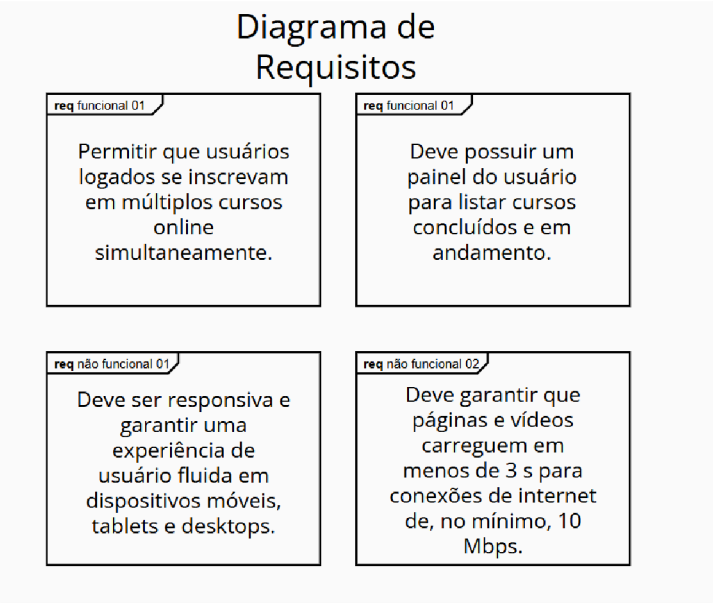
Estávamos analisando a plataforma da Alura, pensando em seu **ciclo de vida**, ainda no estágio de concepção, e mencionamos o uso de uma linguagem gráfica chamada **UML**. Quando iniciamos o processo de concepção de uma plataforma ou software em geral, definimos os **requisitos**. Esses requisitos nos ajudam a estabelecer as **regras de negócio**, que são todas as funcionalidades esperadas do software.

Analisando requisitos

Para modelar bem todo o sistema usando uma linguagem gráfica, o primeiro passo é definir os requisitos e as regras de negócio. Em geral, não possuímos dentro da UML um diagrama específico para elaborar os requisitos.

Por isso, utilizamos um diagrama de blocos para criar uma série de cartões para os requisitos, considerando o caso de uma

plataforma educacional. Dividimos os requisitos em duas categorias: funcionais e não-funcionais.



Os **requisitos funcionais** estão relacionados às funcionalidades que serão implementadas na plataforma. Já os **não-funcionais** estão mais ligados à experiência da pessoa usuária. Por exemplo, a plataforma deve ser responsiva e garantir uma experiência fluida em dispositivos móveis, tablets e desktops. Além disso, deve garantir um carregamento de menos de 3 segundos para uma determinada velocidade de conexão com a internet.

> Em todos as descrições desses requisitos, buscamos ser **concisos e claros**. Além disso, um requisito deve ser **testável ou verificável**.

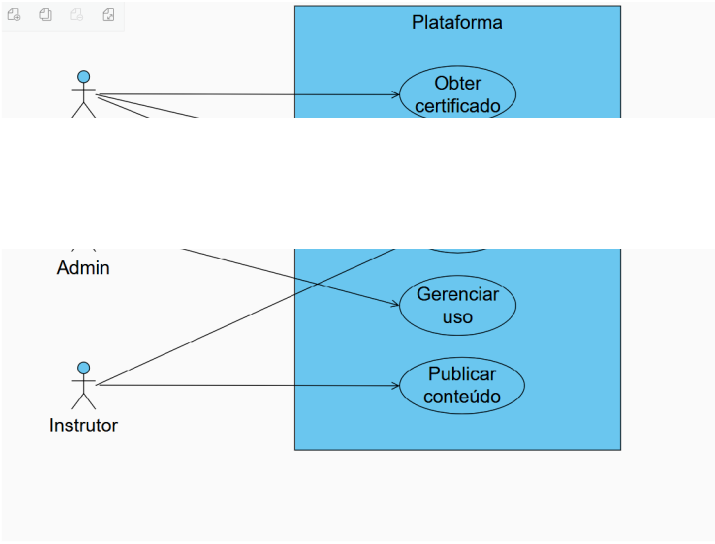
Em outras palavras, devemos conseguir entrar na plataforma e testar se a funcionalidade foi implementada e se o requisito não-funcional está sendo atendido. Um conjunto de requisitos ou regras de negócio opera como um **contrato** entre nós, pessoas desenvolvedoras, e *stakeholders*, que são as partes interessadas no software. Cada requisito é como uma cláusula que devemos cumprir.

No processo de desenvolvimento, os requisitos ou regras que definimos são nossa bússola para guiar todo o processo de desenvolvimento. A partir deles, podemos construir diagramas da UML para discutir funcionalidades e também as camadas e componentes da plataforma.

Conhecendo diagrama de casos de uso

Vamos começar pelo **diagrama de casos de uso**. Este diagrama é comum na descrição de um software e descreve alguns casos de uso, ou seja, **como a plataforma será utilizada** pelos seus atores.

Vamos conferir um exemplo simples de diagrama de caso de uso da mesma plataforma de cursos online:



Os atores são as **pessoas usuárias**, com **diferentes perfis**. Nessa plataforma, temos a figura de **admin**, da pessoa instrutora e da pessoa estudante. Essas diferentes pessoas realizam tarefas distintas.

Dentro do retângulo à direita onde descrevemos as funcionalidades a que cada ator terá acesso na plataforma, utilizamos uma **linguagem concisa e clara**. Por exemplo, a pessoa estudante poderá obter o certificado, realizar o curso e fazer o login. Todos poderão fazer o login. Além disso, o admin poderá gerenciar o uso da plataforma, enquanto a pessoa instrutora poderá publicar conteúdo.

Para criar um bom diagrama de casos de uso, com detalhes e aspectos do funcionamento de cada caso, incluindo funcionalidades internas, precisamos consultar a documentação oficial da UML para entender como usar os **recursos gráficos** disponíveis. No caso da ferramenta

Visual Paradigm, esses recursos ficam listados no painel lateral esquerdo.

> Cada recurso gráfico tem um significado específico. Por isso, precisamos conhecê-los para poder montar cada diagrama da maneira adequada.

É importante destacar que os requisitos são nossa bússola no processo de desenvolvimento e podem ser entendidos como **cláusulas de um contrato**. No entanto, ao longo da vida de uma plataforma, esses requisitos invariavelmente passarão por **modificações**. Por exemplo, na Alura, agora temos um chatbot inteligente para ajudar na trajetória de aprendizagem. Antes, há dois ou quatro anos, isso não era um requisito da plataforma. Hoje, já é.

> À medida que a tecnologia evolui, precisamos **atender a novos requisitos**, pois estão intimamente relacionados às expectativas das pessoas usuárias e ao dinamismo do mercado em que a solução está inserida

Por isso, é fundamental ter um processo contínuo de controle e discussão dos requisitos, entendendo que eles mudam. A documentação do software é essencial para

entender em que ponto precisamos implementar uma nova funcionalidade ou modificação.

Próximos passos

No diagrama de caso de uso, obtemos uma visão geral dos principais componentes inseridos na solução. Tanto as pessoas desenvolvedoras, quanto as partes interessadas terão uma ideia geral da estrutura da plataforma. A partir daí, podemos decidir sobre a arquitetura, estrutura e componentes a serem utilizados.

Além disso, temos outros diagramas, como o **diagrama de classes**. Vamos explorar mais essa diferença entre os diagramas para entender como podemos usar cada um para representar nossa solução.

Transcrição

Estamos explorando o estágio de concepção de uma aplicação, pensando no contexto de desenvolvimento. Nesse processo, estamos lidando com a parte de **modelagem da aplicação**. O caso que estamos tratando mais especificamente é de uma plataforma de cursos online, como a Alura, que está mais próxima da nossa realidade.

Já construímos o diagrama de casos de uso e identificamos os atores, além de como interagem com a plataforma. No entanto, temos outros diagramas que nos ajudam a descrever a própria **estrutura da aplicação**.

Em geral, podemos dividir a UML em duas linhas de diagramas:

- **Diagramas estruturais**: como se estivéssemos pensando na construção de uma casa, com os principais elementos da estrutura;
- **Diagramas de comportamento**: ou seja, como essa estrutura vai interagir com a

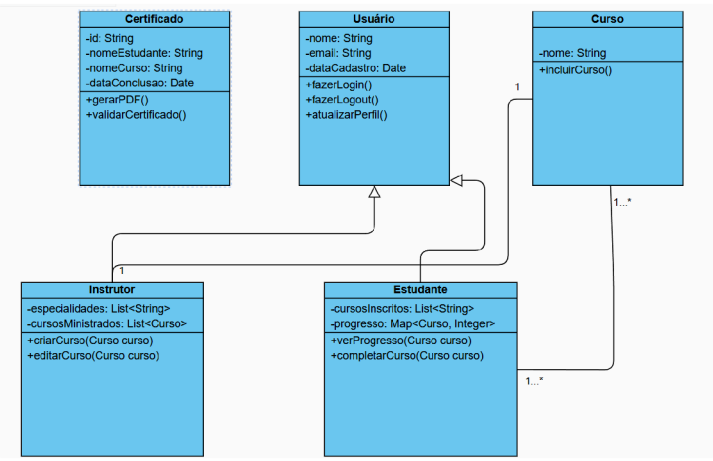
pessoa usuária, que tipo de funcionalidade ela terá e quais eventos ocorrerão dentro da estrutura, por exemplo, quando alguém clica em um botão em uma interface da plataforma.

Podemos começar com um diagrama que representa a estrutura, chamado de **diagrama de classes**. Como o próprio nome sugere, ele presume que nossa aplicação utilizará o **paradigma de programação orientada a objetos**. A partir desse paradigma, vamos construir as principais classes que utilizaremos na aplicação.

Modelando diagrama de classes

Vamos utilizar um editor UML chamado Visual Paragidm, que é gratuito dentro de certos limites, mas você pode escolher a ferramenta que lhe for mais conveniente.

Nesse diagrama de classes, já incluímos algumas classes, como `Certificado`, `Usuário`, `Curso`, `Instrutor` e `Estudante`.



A classe `Usuário` tem setas relacionadas a `Instrutor` e `Estudante` porque é uma

poderíamos incluir outra classe, como `Admin`.

Cada classe é presentada por um retângulo que possui o nome que demarca a classe, seguido de uma seção para os atributos e outra para as operações dessa classe. Por exemplo, todo `Usuário` tem um nome, um e-mail e uma data de cadastro na plataforma. As operações (ou métodos) dentro desse `Usuário` incluem fazer login, fazer logout e atualizar perfil.

Já as **subclasses**, `Instrutor` e `Estudante`, terão diferentes atributos e operações. O `Instrutor` terá uma lista de especialidades e de cursos ministrados como seus atributos e, como métodos, poderá criar e editar cursos. A classe `Instrutor` está conectada à classe `Curso`.

Na classe `Curso`, poderíamos adicionar um atributo. Todo curso tem um nome, que geralmente é uma *string*. Também poderíamos adicionar uma operação ou método para, por exemplo, incluir um curso.

Dentro de cada uma dessas classes, podemos descrever os métodos necessários para o funcionamento da plataforma. Lembrando que, ao pensar no funcionamento, seguimos a lista de regras ou requisitos para a plataforma como parâmetro.

Note que temos **diferentes tipos de associação** entre essas classes. A associação entre `Usuário` para com `Instrutor` e `Estudante` é uma relação de **generalização**, pois são subclasses da superclasse `Usuário`.

A relação entre `Curso` e `Estudante` é representada por uma linha que integra as duas classes e a relação `1...*` (um para muitos) - isso indica que um estudante pode fazer vários cursos simultaneamente, ao mesmo tempo que um curso pode ter vários estudantes simultaneamente. Essa associação é chamada de **um para muitos**. Da mesma forma, se tivéssemos a relação de `Certificado` para `Estudante`, teríamos o mesmo tipo de associação.

Conclusão

Com esse diagrama de classes, conseguimos estruturar todo o sistema com base na orientação a objetos. Existem outros diagramas, como:

- **Diagrama de atividade**: descreve uma sequência de atividades dentro da aplicação, ou seja, o comportamento da aplicação;
- **Diagrama de estado**: descreve os estados pelos quais a aplicação passará - sempre pensando no momento da interação.

No diagrama de casos de uso, descrevemos as principais funcionalidades e o que haverá dentro da plataforma em termos de comportamento. Usando esses outros diagramas, construímos outras perspectivas do que ocorrerá dentro da aplicação.

Nessa fase de modelagem, queremos ter uma **visão sistêmica** do funcionamento e da estrutura da plataforma. Assim, podemos decidir que tipo de **linguagem e componentes** utilizaremos, bem como definir a **arquitetura do software**.

Imagine que moramos em uma família de quatro pessoas e queremos construir uma casa de acordo com determinado orçamento e terreno. Queremos definir como essa casa será construída para atender às necessidades da família. É uma boa analogia para a construção de um software.

Temos uma demanda a atender, descrita pelos requisitos, e desenhamos representações desses sistemas prováveis. E também discutimos com as pessoas interessadas no desenvolvimento do software para entender qual o melhor caminho a seguir.

Agora que entendemos como modelar para atuar bem nesse estágio de concepção de uma aplicação, é hora de avaliar as arquiteturas que podemos utilizar para construir um software.

Para saber mais: diagramas UML

A Unified Modeling Language (UML) inclui uma variedade de diagramas, cada um com um propósito específico. Vamos explorar os principais diagramas e como eles se aplicam a uma plataforma de cursos online, como a Alura, para facilitar a compreensão da estrutura e do comportamento do sistema.

Diagrama de Casos de Uso

Permite identificar as interações entre o sistema e seus usuários. No caso de uma plataforma de cursos online, os principais atores podem ser estudantes, instrutores, administradores e o sistema em si. O diagrama de casos de uso permite visualizar as principais funcionalidades, como:

- Estudante realizando login, inscrevendo-se em cursos e obtendo certificados.
- Instrutor publicando conteúdo e gerenciando seus cursos.
- Administrador gerenciando usuários e cursos.

Diagrama de Classes

Esse é um diagrama que nos ajuda a representar a estrutura estática de um sistema, descrevendo as classes e seus relacionamentos. Para a plataforma de cursos online, as classes poderiam incluir:

- Usuário (classe base), com suas subclasses Estudante e Instrutor.
- Curso, relacionado ao Instrutor e ao Estudante.
- Certificado, gerado para o Estudante ao concluir um curso.

Com o diagrama de classes temos uma visão abrangente de como os diferentes componentes do sistema interagem e se estruturam.

Diagrama de Sequência

Apresenta como os objetos interagem entre si ao longo do tempo, detalhando a ordem das mensagens trocadas. Na plataforma de cursos online, podemos usar esse diagrama para descrever o fluxo de ações, como o processo de inscrição em um curso e a geração de certificado.

Diagrama de Atividades

Descreve o fluxo de trabalho de um processo, como a inscrição de um estudante em um curso,

acompanhando as ações do usuário até a conclusão da tarefa. É um diagrama bastante eficaz para mapear fluxos de interação e garantir que todas as etapas necessárias sejam atendidas.

Diagrama de Estados

Indica os diferentes estados pelos quais um objeto pode passar durante seu ciclo de vida. Por exemplo, o estado de um curso pode variar entre Em andamento, Concluído ou Descontinuado, e o certificado de um aluno pode ter estados como Em emissão ou Emitido.

Diagrama de Componentes

O diagrama de componentes descreve a estrutura física do sistema, focando na organização dos módulos e como eles interagem. Para uma plataforma online, ele pode ser usado para mostrar como os diferentes componentes de software (como o backend, banco de dados e frontend) se comunicam.

Diagrama de Implantação

Descreve a configuração física de um sistema, como servidores e redes. Para uma plataforma de cursos online, ele pode ser utilizado para representar a infraestrutura necessária para hospedar a aplicação, como servidores na nuvem e seus serviços.

Esses diagramas são fundamentais para a documentação e comunicação eficaz entre as equipes técnicas e não técnicas, assegurando que todos os envolvidos no desenvolvimento do software compreendam claramente os requisitos do sistema e seu funcionamento.

OFERTA ESPECIAL: ATÉ
35% OFF ACABA EM 20/03

APROVEITE!

07 : 04 : 08
HORAS MIN SEG

CUPOM: **SEMANADOCONSUMIDOR**

OFERTA ESPECIAL: ATÉ
35% OFF ACABA EM 20/03

APROVEITE!

07 : 04 : 08
HORAS MIN SEG

CUPOM: **SEMANADOCONSUMIDOR**

Camila Pessôa
04/02/2024
Atualizado 3 dias atrás

COMPARTILHE

Confira neste artigo:

- Introdução
- O que é arquitetura de software
- Qual a melhor arquitetura de software?
- Conclusão
- Referências

Introdução

OFERTA ESPECIAL: ATÉ
35% OFF ACABA EM 20/03

APROVEITE!

07 : 04 : 08
HORAS MIN SEG

CUPOM: **SEMANADOCONSUMIDOR**

facilitar a representação de soluções em softwares.

Pensando nisso, este artigo vai apresentar alguns dos padrões de arquitetura de software já consolidados no mercado para que você tenha um direcionamento nos seus estudos. Vamos lá?

Conhecer padrões arquiteturais é o que diferencia quem apenas desenvolve de quem constrói soluções escaláveis.
Na **Semana do Consumidor da Alura**, você garante até **35% de desconto** para estudar por 2 anos e aprofundar seus conhecimentos em arquitetura e boas práticas.
Não perca essa oportunidade, escolha o melhor plano e avance na sua carreira!

O que é arquitetura de software

Para entendermos os padrões arquiteturais, primeiro é necessário compreendermos o que é **arquitetura de software**.

As apropriações de nomenclaturas do mundo real são comuns no universo de tecnologia, e não poderia ser diferente com o termo **arquitetura**. No mundo físico, profissionais de arquitetura constroem projetos e desenhos de casas,

Transcrição

Olá, o instrutor da Alura, **Lucas Mata**, te dá as boas-vindas ao curso sobre o **ciclo de vida de aplicações**.

> **Audiodescrição:** Lucas Mata se descreve como uma pessoa branca.

> arrendondada e camiseta azul-escura. Ao fundo, estúdios da Alura com iluminação azulada. À direita, uma estante com decorações.

Este curso é indicado para quem deseja ter uma **visão sistêmica do processo de desenvolvimento de software** - desde a etapa de **concepção**, quando temos uma ideia de solução para resolver um problema, até a **implantação no ambiente de produção**, onde o software será efetivamente utilizado.

Sendo assim, é indicado para pessoas desenvolvedoras, lideranças, gestões,

profissionais que atuam na área de operações de infraestrutura e também para entusiastas de tecnologia em geral.

O que vamos aprender?

Nós vamos aprender a **modelar uma aplicação utilizando diagramas do UML** (*Unified Modeling Language* ou Linguagem de Modelagem Unificada, em português) e como podemos usar esses diagramas para nos comunicarmos bem com as diferentes partes interessadas no processo de desenvolvimento de software.

Vamos explorar também diferentes **abordagens arquiteturais**, como microsserviços, monolítica, *pipes and filters* (tubos e filtros), que podemos utilizar para construir nossa aplicação. Além disso, vamos explorar diferentes **padrões de design** para construir bem cada componente que fará parte do nosso software.

Assim, analisaremos os principais componentes de uma aplicação, como *back-end*, *front-end* e banco de dados, bem como os princípios que fornecem diretrizes de boas práticas para a construção do código, como o **SOLID**.

Além de aprender a selecionar **linguagens e *frameworks*** para o desenvolvimento, também entenderemos a importância dos **testes** no processo de liberação de novas versões, módulos e sessões de uma aplicação e da adoção de **metodologias ágeis** para o processo de desenvolvimento.

Uma vez definidos os requisitos, a linguagem e o *framework*, precisamos nos organizar como equipe para atender às expectativas das partes interessadas nesse processo de desenvolvimento e construir uma solução com qualidade, sempre com testes e validações.

Por fim, aprenderemos como **implantar essa aplicação no ambiente de produção**. Para isso, exploraremos estratégias de *deploy*, diferentes tipos de infraestrutura - como *on-premises* (local) e *cloud* (nuvem) - e também diferentes formas de implantação, seja utilizando instâncias, *containers* ou ferramentas de orquestração, como Kubernetes.

Iremos aprender tudo isso através da análise de cada **estágio do ciclo de vida da aplicação**, passando pelas seguintes etapas:

1. Concepção;
2. Arquitetura e design;
3. Desenvolvimento;

4. Testes;
5. Implantação.

Nesse curso, não abordaremos as últimas três etapas de monitoramento, manutenção e atualização e evolução.

Abordaremos cada etapa de maneira significativa e contextualizada, trazendo casos reais para entender como aplicar cada ferramenta, abordagem e técnica.

> Aproveite todos os recursos disponíveis na plataforma. Além do **fórum**, temos também a **comunidade no Discord**.

Nos encontramos nos próximos vídeos para iniciar essa jornada de aprendizado pelo mundo do software.

Transcrição

Quando pensamos em software, geralmente a primeira ideia que nos vem à mente é uma série de linhas de código e módulos nos quais implementamos funcionalidades de uma aplicação.

De fato, desenvolver um software envolve muita programação, mas devemos lembrar que esse software não existirá apenas para nós, pessoas desenvolvedoras. Ele estará inserido em um contexto, podendo ser em uma farmácia, padaria, indústria ou até mesmo no smartphone das **pessoas usuárias**. Atualmente, interagimos constantemente com aplicativos móveis, que também são exemplos de software.

> Software é uma solução para as pessoas em geral!

É comum pensar que o desenvolvimento de software está altamente relacionado ao campo das ciências exatas. De fato,

utilizamos uma série de conhecimentos e

software e áreas afins. No entanto, o processo de construir um software não é essencialmente determinístico.

Não conseguimos, *a priori,* inserir uma série de funcionalidades esperadas e aplicar uma teoria matemática, fórmula ou modelo que nos permita obter o software perfeito para aquelas funcionalidades que pretendemos implementar. Portanto, é um processo que envolve **criatividade** e habilidade da pessoa programadora, utilizando uma linguagem, um *framework* ou um conjunto de ferramentas nesse processo de desenvolvimento.

Como lidamos com esse processo de desenvolvimento de um software?

Compreendendo o ciclo de vida

Imagine que recebemos uma demanda para desenvolver um software ERP (*Enterprise Resource Planning* ou Planejamento de Recursos Empresariais, em português) para um mercado ou pequeno negócio no nosso bairro. Como lidamos com esse processo? Começamos implementando diretamente? Não. Vamos considerar um caso concreto

próximo a nós, que é a plataforma da Alura, onde estamos fazendo este curso.

Na plataforma, temos uma série de funcionalidades, incluindo a inteligência artificial da Luri, que nos permite interagir e obter conteúdos extras, além dos artigos, vídeos, cursos e formações disponíveis.

Ao longo dos mais de 10 anos de existência da plataforma no Brasil, começamos com funcionalidades mais básicas, assim como as demandas de conteúdo, que eram mais voltadas para o desenvolvimento web e programação em Java e C, linguagens mais comuns na época do lançamento da plataforma. Atualmente, temos uma variedade de conteúdos diversificados para todos os públicos, o que possibilita uma formação continuada.

Quando pensamos na trajetória de uma plataforma como a Alura ou de um aplicativo móvel que precisa ser atualizado, seja por mudanças no Android ou por vulnerabilidades detectadas, consideramos um **ciclo de vida da aplicação**. Esse ciclo de vida começa na concepção da plataforma, ou seja, nessa solução que vamos construir em forma de software.

> O **ciclo de vida** é composto por uma série de estágios pelos quais uma

aplicação passará ao longo de sua vida útil.

O primeiro estágio é a **concepção**, onde não estamos programando de fato, mas, sim, **desenhando a solução**.

Devemos pensar no software sempre como uma solução para outras pessoas. Por isso, no processo de concepção, devemos interagir bastante com as pessoas envolvidas, ou seja, as **partes interessadas**, conhecidas na linguagem de negócios e gestão como ***stakeholders***. Muitas vezes, não conversamos diretamente com as pessoas usuárias, mas sim com *stakeholders* e parceiros, para entender o que buscam em termos de solução de software. A partir dessa conversa, definimos os **requisitos**.

O que é um requisito? Vamos pensar no caso da Alura. Ao interagir com a plataforma, podemos nos inscrever em cursos para criar uma primeira aplicação tanto com C# quanto com Python. É possível fazer os cursos dessas duas linguagens de maneira simultânea. Ao final de cada curso, podemos obter o certificado, compartilhar e incluir em nossa pasta de conquistas profissionais.

Tudo isso são requisitos da plataforma. Um dos requisitos é permitir que uma pessoa se

inscreva simultaneamente em diferentes cursos, sem limitar a jornada de aprendizagem. Quando a pessoa conclui um curso, há uma funcionalidade para gerar um certificado que ficará permanentemente disponível em seu perfil.

Além disso, temos requisitos relacionados à interação da pessoa usuária com a plataforma, como **usabilidade e navegabilidade**. Por exemplo, ao clicar para postar uma dúvida no fórum, esperamos uma resposta rápida da plataforma. Ninguém quer esperar mais de 10 segundos para carregar uma página. Esse tipo de parâmetro, relacionado ao tempo de resposta e latência da plataforma, é um requisito operacional importante para garantir uma **boa experiência de uso** na Alura ou em qualquer outra aplicação web.

> Os requisitos são levantados no início do processo de concepção da plataforma. A partir desses requisitos, que foram definidos em conjunto com as partes interessadas, construímos uma série de **diagramas descritivos**.

Devemos construir diagramas para descrever a solução de forma gráfica. No entanto, em vez de modelar essa solução por meio de desenhos livres que apenas nós compreenderíamos, usamos uma

linguagem unificada que permite que diferentes pessoas desenvolvedoras e parceiras compreendam o que estamos desenhando.

> Essa linguagem gráfica padrão é chamada de **UML** (*Unified Modeling Language* ou Linguagem Unificada de Modelagem).

A UML é bem conhecida, começou a ser adotada no final dos anos 90 e é utilizada até hoje. Existem diversas documentações em vários sites, como o da IBM, que orientam sobre como criar diagramas de classe (https://www.ibm.com/docs/pt-br/dmrt/9.5?topic=diagrams-creating-class) e diagramas de casos de uso, descrevendo as principais formas de uso da plataforma ou solução que vamos construir.

Além disso, existem uma série de editores de diagramas online, que permitem construir diagramas de forma simples e rápida - sem precisar instalar nada no computador. Basta procurar por "UML ferramentas" em um motor de pesquisa, como o Google.

Próximos passos

A UML é uma linguagem interessante no processo de modelagem para conceber e discutir como a solução em forma de software será implementada. Vamos explorar mais sobre o mundo da modelagem de aplicações na sequência.

04

Para saber mais: o que é UML?

A **Unified Modeling Language (UML)** é uma linguagem de modelagem padronizada, essencial para projetar e visualizar sistemas de software. Como uma linguagem, seu principal objetivo é facilitar a comunicação entre as diferentes partes envolvidas no desenvolvimento, manutenção e operação de uma aplicação, como pessoas desenvolvedoras, analistas de negócios e stakeholders não técnicos.

Em um projeto de software, é comum que existam diversas partes interessadas, cada uma com um nível diferente de conhecimento técnico e uma perspectiva. A UML atua como uma ponte de comunicação visual, simplificando o entendimento do sistema e garantindo que todos os envolvidos estejam alinhados. Por exemplo, uma representação de caso de uso pode esclarecer as funções que o sistema precisa cumprir para atender às necessidades do usuário final.

A UML inclui uma variedade de diagramas que servem a diferentes propósitos no desenvolvimento de software. Entre os mais comuns estão:

- Diagramas de casos de uso: destacam as interações entre atores e o sistema;
- Diagramas de classes: representam a estrutura estática e os relacionamentos das entidades do sistema;
- Diagramas de atividades: detalham o fluxo de processos e decisões do sistema;
- Diagramas de sequência: mostram a troca de mensagens entre objetos em uma ordem cronológica.

Cada um desses diagramas oferece uma visão específica do sistema, permitindo uma compreensão mais abrangente e detalhada do funcionamento e da arquitetura da aplicação. Falaremos mais profundamente sobre eles e suas diferenças ao longo desta aula.

Como criar diagramas UML?

Podemos usar diversas ferramentas gráficas para criação e visualização de diagramas UML, tornando o processo de modelagem de aplicações mais ágil e acessível. Vamos indicar aqui algumas opções de ferramentas:

Lucidchart (https://www.lucidchart.com/pages/?): Ferramenta online que permite criar diagramas de fluxo, UML e outros tipos de visualizações. Possui integração com plataformas como Google Drive, facilitando o trabalho colaborativo.

Visual Paradigm (https://www.visual-paradigm.com/): Ferramenta amplamente adotada no mundo corporativo para o design de software. Possui suporte a todos os tipos de diagramas UML e outras funcionalidades, como geração de código.

Draw.io (diagrams.net) (https://app.diagrams.net/): Ferramenta gratuita e acessível para criar diagramas UML de forma simples e rápida. Permite exportar os diagramas em vários formatos.

PlantUML (https://plantuml.com/): Ferramenta baseada em texto para gerar diagramas UML a partir de uma linguagem de marcação simples. Ideal para quem prefere escrever código para criar diagramas. Pode ser integrada com diversas IDEs, como IntelliJ e Visual Studio Code.

StarUML (https://staruml.io/): Ferramenta de modelagem UML com uma interface simples e suporte para criação de diagramas como classes, casos de uso e sequência.

Enterprise Architect (https://sparxsystems.com/products/ea/): Uma solução mais avançada para profissionais que necessitam de funcionalidades complexas, como integração com outras ferramentas de gerenciamento de projetos e suporte a grandes equipes de desenvolvimento.

Recomendamos explorar e analisar cada uma das ferramentas para selecionar aquela que melhor atende às suas necessidades de uso!